ЗАДАНИЕ

ТЗ: Мини-проект «Учёт заявок техподдержки» на Django

1. Идея

Внутренняя админка для небольшой IT-компании: сотрудники (операторы) ведут заявки от клиентов. Данные о клиентах и заявках периодически приходят в Excel-выгрузках из старой CRM — их надо заливать прямо из админки, а не руками.

2. Функциональные требования

2.1. Модели (приложение helpdesk)

Client — клиент
- name — CharField(max_length=150), название клиента или ФИО.
- email — EmailField, unique=True, используется как ключ при импорте.
- phone — CharField(max_length=20), blank=True.
- company — CharField(max_length=150), blank=True.
- created_at — DateTimeField(auto_now_add=True).

Метод:
- __str__ → возвращает name.

2. Ticket — заявка
- external_id — CharField(max_length=30), unique=True, ID из старой CRM. Ключ для обновления при импорте.
- client — ForeignKey → Client, on_delete=PROTECT, related_name="tickets".
- subject — CharField(max_length=200), тема заявки.
- description — TextField, blank=True, подробное описание.
- priority — CharField с choices: low, normal, high, critical; default=normal.
- status — CharField с choices: new, in_progress, waiting, closed; default=new.
- assignee — ForeignKey → User, null=True, blank=True, on_delete=SET_NULL, related_name="assigned_tickets".
- created_at — DateTimeField, дата создания заявки (может прийти из Excel, не auto_now_add).
- closed_at — DateTimeField, null=True, blank=True, проставляется автоматически при переходе в closed.

Свойства / методы:
- __str__ → f"#{external_id} {subject}".
- save() — переопределён: если status == "closed" и closed_at пуст → ставит timezone.now().
- is_overdue (property) — True, если priority=critical и статус не closed и прошло больше N часов с created_at (опционально).

3. Comment — комментарий к заявке
- ticket — ForeignKey → Ticket, on_delete=CASCADE, related_name="comments".
- author — ForeignKey → User, null=True, on_delete=SET_NULL.
- text — TextField, текст комментария.
- created_at — DateTimeField(auto_now_add=True).

Метод:
- __str__ → первые 50 символов текста.

4. ImportLog — журнал импортов
- user — ForeignKey → User, null=True, on_delete=SET_NULL, кто загрузил файл.
- file_name — CharField(max_length=255), имя загруженного файла.
- uploaded_at — DateTimeField(auto_now_add=True).
- total_rows — PositiveIntegerField, всего строк в файле.
- created_count — PositiveIntegerField, создано заявок.
- updated_count — PositiveIntegerField, обновлено заявок.
- skipped_count — PositiveIntegerField, пропущено строк.
- errors — JSONField(default=list), список строк с описанием ошибок по строкам.

Метод:
- __str__ → f"{file_name} — {uploaded_at:%Y-%m-%d %H:%M}".

5. Сводная таблица связей
- Client 1 → N Ticket (через Ticket.client).
- Ticket 1 → N Comment (через Comment.ticket).
- User 1 → N Ticket (через Ticket.assignee).
- User 1 → N Comment (через Comment.author).
- User 1 → N ImportLog (через ImportLog.user).

6. Что НЕ забыть при описании моделей
- Для Ticket.priority и Ticket.status — вложенные TextChoices (класс внутри класса) и человекочитаемые метки (get_status_display()).
- Для Ticket — Meta.ordering = ["-created_at"] и Meta.indexes по status и assignee.
- Для Client.email — db_index=True (используется в импорте).
- Для ImportLog — Meta.ordering = ["-uploaded_at"].
- on_delete=PROTECT у Ticket.client, чтобы нельзя было удалить клиента с заявками.
- verbose_name / verbose_name_plural на русском для всех моделей — пригодится в админке.

3. Что нужно реализовать в админке

3.1. TicketAdmin
- Колонки: external_id, subject, client, priority (цветной бейдж), status (цветной бейдж), assignee, created_at.
- list_filter: priority, status, assignee, дата создания.
- search_fields: subject, client__name, external_id.
- Inline: CommentInline (TabularInline) — комментарии прямо на странице заявки.
- readonly_fields: created_at, closed_at.
- Автологика: если статус меняется на closed, a closed_at пуст — проставить now() (через save_model).
- Action assign_to_me — назначить выбранные заявки на текущего юзера.
- Action close_tickets — массово закрыть с простановкой closed_at.
- Action export_selected — выгрузить выбранные заявки в .xlsx с подсветкой приоритета.

3.2. Кастомная страница «Импорт заявок из Excel»

Доступна по кнопке на change_list (через get_urls() и переопределение шаблона).

Логика импорта:
1. Пользователь загружает .xlsx.
2. Парсим колонки: external_id, client_email, client_name, subject, description, priority, status, created_at.
3. Если клиента с таким email нет — создаём.
4. Если заявка с таким external_id есть — обновляем (updated_count++).
5. Если нет — создаём (created_count++).
6. Битые строки (нет external_id / subject, неизвестный priority, кривая дата) — в errors, skipped_count++.
7. По завершении — создаём ImportLog и показываем страницу-отчёт с таблицей: сколько добавлено / обновлено / пропущено и почему.